

Windows Forensics Toolkit –V1

Python-Based Security Event and Registry Analysis Tool

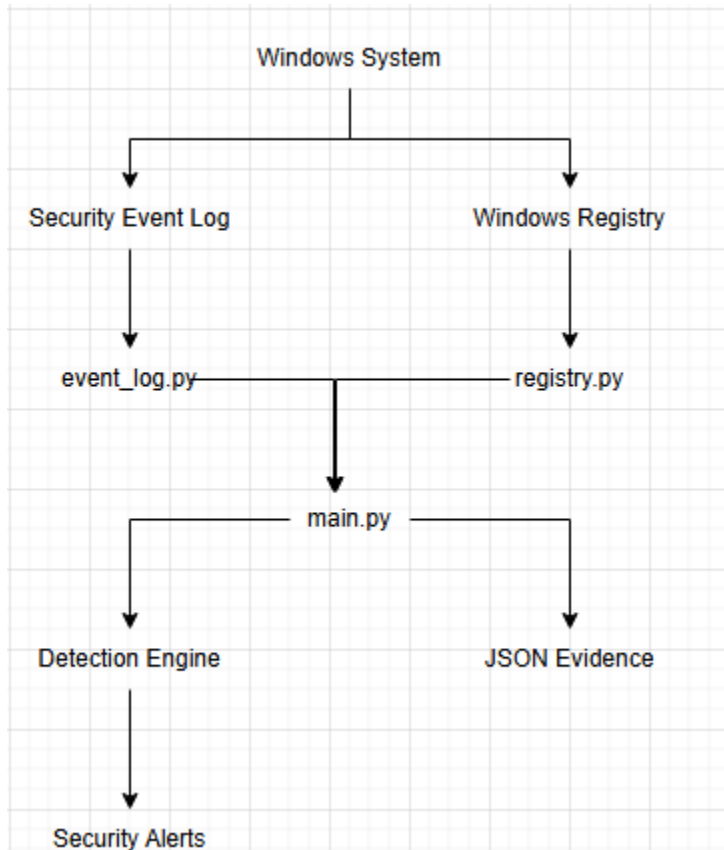
1. Introduction

Soc Analysts and forensic investigators need to examine large amounts of Windows telemetry to identify potentially suspicious behavior. Manually looking over Event Viewer and Registry locations can be time-consuming.

2. Project Objective

- Automatically collect selected Windows Security events
- Change raw Windows XML into formatted forensic records
- Export evidence as JSON
- Detect repeated authentication failures
- Examine Registry Run keys for persistence
- Flag potentially suspicious startup entries

3. System Design



4. Windows Event Log Collection

Windows Event Logs give a detailed record of activities happening on a Windows System, including user authentication, process execution and security-related events. These logs are very useful during security monitoring and digital forensic investigation because they allow an analyst to identify and reconstruct activity that has happened on a system.

The Windows Forensics Toolkit was made to automatically collect a selection of security-relevant events from the Windows Security log. The following Event IDs were selected for the 1st version of the toolkit.

Event ID	Event	Relevance
4624	Successful Logon	Identifies successful authentication activity and can assist in determining when an account accessed the system

4625	Failed Logon	Identifies unsuccessful authentication attempts and can be used to identify repeated login failures or potential brute force activity
4634	Account Logoff	Records when a logon session ends and can assist in reconstructing user session
4648	Logon Using Explicit Credentials	Shows that credentials were explicitly supplied, which can be useful when investigating account usage
4688	Process Creation	Records the creation of a new process and can provide evidence of programs or commands executed on the system.
1102	Audit Log Created	Records when the Windows Security audit log has been cleared. This is relevant because clearing security logs may be associated with attempts to remove evidence of activity.

4.1 Event Collection Method

The toolkit collects these events using PowerShell (**Get-WinEvent**) command. Python's subprocess module is used to execute the PowerShell command from within the program.

The selected Event IDs are defined in the (**event_log.py**) module:

```
EVENT_IDS = [4624, 4625, 4634, 4648, 4688, 1102]
```

The program makes a PowerShell query requesting events from the Windows Security log:

```
Get-WinEvent -FilterHashtable @{LogName='Security'; Id={event_ids_string}} -MaxEvents 100 |
```

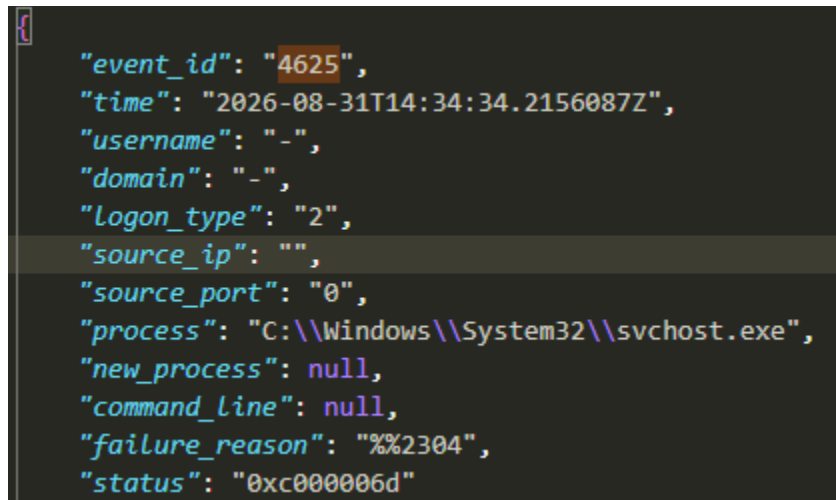
A limit of 100 events was used to keep the amount of collected data manageable while testing the toolkit. Rather than relying on the formatted text normally shown by the Windows Event Viewer, each retrieved event is converted into XML using the (**ToXML()**) method. This gives structured fields that can be made individually by the toolkit.

4.2 Initial Parsing

The Collection functionality was initially tested using EVENT ID 4625,

Which represents a failed logon attempt. Failed logon attempts were deliberately generated on the test system, and the toolkit was then executed to show whether the corresponding events could be retrieved.

The test successfully returned Event ID 4625 records containing information of the timestamp, target account, logon type, source network address, authentication information and failure status.



```
{
  "event_id": "4625",
  "time": "2026-08-31T14:34:34.2156087Z",
  "username": "-",
  "domain": "-",
  "logon_type": "2",
  "source_ip": "",
  "source_port": "0",
  "process": "C:\\Windows\\System32\\svchost.exe",
  "new_process": null,
  "command_line": null,
  "failure_reason": "%%2304",
  "status": "0xc000006d"
}
```

Figure 1: Windows Event ID 4625 records retrieved by the toolkit

This confirmed that the Python program could successfully communicate with the Windows Event Log infrastructure and retrieve security events for subsequent forensic analysis.

4.3 Security Considerations

Access to some Windows Security logs may require elevated privileges. During development, the toolkit was therefore executed with appropriate permissions where necessary. The tool performs read operations against the Security log and does not modify or delete the collected Windows events.

5. Event Parsing and Evidence Collection

After the Windows Security events are collected, the raw event data must be converted into a format that can be easily analyzed by the toolkit. Windows events contain a large amount of information, and working directly with the formatted output displayed in Event Viewer would make automated analysis difficult.

To address this, the toolkit retrieves each event in **XML format** and uses Python to extract the individual fields that are relevant to a security investigation. The extracted information is then converted into structured Python records and saved as JSON.

5.1 Parsing Windows Event XML

The (**event_logs.py**) module uses Python's xml.etree.ElementTree library to process the XML returned during event collection.

These namespaces are required because Windows Event XML uses Microsoft's event schema:

```
namespace = {  
    "event": "http://schemas.microsoft.com/win/2004/08/events/event"  
}
```

The toolkit first extracts basic information from the System section of each event, including the Event ID and the time at which the event was created.

```
# Get basic event information  
Event_id = root.find("event:System/event:EventID", namespace).text  
time_created = root.find("event:System/event:TimeCreated", namespace).attrib["SystemTime"]
```

The (EventData) section is then processed to obtain additional information associated with the event.

```
#Get EventData fields  
event_data = {}  
  
for data in root.findall("event:EventData/event:Data", namespace):  
    field_name = data.attrib.get("Name")  
    field_value = data.text  
  
    event_data[field_name] = field_value
```

This allows the toolkit to access individual Windows event fields by their names rather than attempting to analyze a large human readable message

5.2 Creation of Forensic Records

Once the XML has been parsed, selected information is palced into a Python dictionary representing a single forensic event.

The toolkit currently extracts fields like:

Field	Purpose
event_id	Identifies the type of Windows event
time	Records when the activity occurred
username	Identifies the target/user account
domain	Identifies the Windows domain
Logon_type	Gives information about the type of authentication
Source_ip	Identifies the source network address where available
Source_port	Records the source network port
process	Identifies an associated process
New_process	Records newly created processes
Command_line	Records available command-line information
Failure_reason	Provides information about authentication failures
status	Contains the WIndows status code associated with an event

```
# Build the forensic record
forensic_event = {
    "event_id": Event_id,
    "time": time_created,
    "username": event_data.get("TargetUserName"),
    "domain": event_data.get("TargetDomainName"),
    "logon_type": event_data.get("LogonType"),
    "source_ip": event_data.get("IpAddress"),
    "source_port": event_data.get("IpPort"),
    "process": event_data.get("ProcessName"),
    "new_process": event_data.get("NewProcessName"),
    "command_line": event_data.get("CommandLine"),
    "failure_reason": event_data.get("FailureReason"),
    "status": event_data.get("Status"),
}
```

Not every Windows Event ID contains all the fields. That's why the `get()` method is used so that the missing fields return an empty/none value rather than causing the program to fail.

This allows different types of security events to be represented using a consistent structure

5.3 Processing Multiple Events

The collection function can return multiple Windows events at once. The toolkit separates the XML data into individual events before sending each event to the parsing function.

```
xml_events = raw_events.split("</Event>")
```

Each valid XML event is then passed to:

```
parse_event(xml_event)
```

And the resulting forensic record is added to an events list.

As a result, multiple different Windows events can be represented using a common structure.

5.4 JSON Evidence Export

After the events have been parsed, they are saved to: `output/security_events.json`

The `(save_events())` function in `main.py` handles this process

```
def save_events(events):  
    os.makedirs("output", exist_ok=True)  
    with open("output/security_events.json", "w") as file:  
        json.dump(events, file, indent=4)
```

JSON was selected because it gives a structured and human readable representation of the collected evidence while also being easy to process using other programs or scripts.

A failed login event would appear like:

```
{
  "event_id": "4625",
  "time": "2026-08-19T09:05:09.7591777Z",
  "username": "-",
  "domain": "-",
  "logon_type": "2",
  "source_ip": "127.0.0.1",
  "source_port": "0",
  "process": "C:\\\\Windows\\System32\\svchost.exe",
  "failure_reason": "%%2304",
  "status": "0xc000006d"
}
```

5.5 Event Summary

The toolkit also generates a summary showing how many events of each selected type were collected.

```
Security events collected: 100

Event Summary
-----
4624 Successful Logons: 83
4625 Failed Logons: 4
4634 Logoffs: 8
4648 Explicit Logons: 5
4688 Process Creation: 0
1102 Security Log Clear: 0

Evidence saved:
output/security_events.json
```

This is good for investigators as it is a quick overview without needing them to manually inspect every JSON record.

5.6 Outcome

The event parsing component transforms raw Windows Security Event XML into structured forensic records that can be stored and analyzed. This provides the foundation for the toolkit detection functionality.

Rather than displaying Windows events, the toolkit can now access specific attributes like the source IP address, target username, timestamp and Event ID. These fields are used in the next stage of the project to show patterns of potentially suspicious activity.

6. Brute Force Detection

Collecting security events gives useful forensic evidence; but manually reviewing large numbers of events can be inefficient. That means, the toolkit includes basic detection logic designed to identify patterns of potentially suspicious authentication activity.

The 1st detection implemented in the toolkit focusses on repeated failed login attempts. Multiple authentication failures against the same account from the same source may indicate an attempted brute-force attack.

Windows records failed authentication attempts using Security Event ID 4625. The toolkit analyses these events and makes an alert when the number of failures reaches a certain threshold.

6.1 Detection Logic

The brute-force detection functionality is made in the **(detections.py)** module. The function accepts the collected security events and a configurable detection threshold:

```
def detect_brute_force(events, threshold = 2):
```

During testing, the threshold was temporarily made reduced to 2 failed attempts so that the detection functionality could be validated without requiring many failed authentication attempts.

A **(defaultdict)** is used to group failed login events:

```
failed_logins = defaultdict(list)
```

The function then examines each collected event and ignores events that are not Event 4625:

```
for event in events:
    if event["event_id"] != "4625":
        continue
```

For each failed login, these information is extracted:

```
source_ip = event.get("source_ip")
username = event.get("username")
timestamp = event.get("time")
```

The source IP address and username are then combined to create a grouping key:

```
key = (source_ip, username)

failed_logins[key].append(timestamp)
```

This is important because the detector does not just count all failed logins on the computer. It attempts to identify repeated failures associated with the same source and target account.

6.2 Alert Generation

After the events have been grouped, the number of failed attempts associated with each source IP and username combination is compared against the configured threshold:

```
for (source_ip, username), timestamps in failed_logins.items():
    if len(timestamps) >= threshold:
```

When the threshold is reached, the toolkit creates an alert containing information useful to an analyst:

```
alert = {
    "type": "Possible Brute Force",
    "severity": "Medium",
    "source_ip": source_ip,
    "username": username,
    "failed_attempts": len(timestamps),
    "timestamps": timestamps
}
```

The generated alert is subsequently displayed by (**main.py**) and can also be saved as structured JSON evidence.

6.3 Testing the Detection

While making this, failed authentication attempts were made on purpose on the Windows system. These events were successfully recorded by Windows as Event ID 4625 and subsequently collected by the toolkit.

The 1st challenge occurred because the generated authentication events contained the loopback address: 127.0.0.1

The original detection logic deliberately excluded loopback addresses:

```
#Ignore events without a useful source IP
if not source_ip or source_ip in ("-", "127.0.0.1", "::1"):
    continue
```

This was made to reduce false positives when detecting **remote** brute-force activity. However, because the controlled test attempts originated locally, the events were excluded from analysis and no alert was generated.

For testing purposes, the condition was temporarily changed to:

```
#Ignore events without a useful source IP
if not source_ip or source_ip in ("-"):
    continue
```

And the threshold was temporarily reduced to 2.

After these changes, repeated failed authentication events associated with the same source and account successfully triggered the brute-force detection.

```
[
  {
    "type": "Possible Brute Force",
    "severity": "Medium",
    "source_ip": "127.0.0.1",
    "username": "-",
    "failed_attempts": 4,
    "timestamps": [
      "2026-08-31T14:34:34.2156087Z",
      "2026-08-31T14:34:33.2316469Z",
      "2026-08-31T14:34:32.1820534Z",
      "2026-08-31T14:34:23.6185515Z"
    ]
  }
]
```

This test shows that the toolkit could successfully progress through the complete detection pipeline.

6.4 False Positives and Detection Limitations

A failed login does not necessarily show malicious activity. Users frequently enter incorrect passwords; applications may use outdated credentials, and Windows services can generate authentication failures during normal operation. For this reason, the toolkit labels the detection as Possible Brute Force.

The current implementation also has an important limitation: it counts matching authentication failures but does not yet require those failures to occur within a specific time window.

For example, five failed attempts occurring within 30 seconds are considerably more suspicious than five failed attempts occurring over several weeks.

6.5 Outcome

The brute-force detection component shows how the toolkit can move beyond simply collecting forensic evidence and begin automatically identifying patterns of potentially suspicious behavior.

By correlating multiple Event ID 4625 records using the source IP address and target username, the toolkit can identify repeated authentication failures and present them to an analyst as a security alert.

Although the current detection is relatively simple, it establishes a framework through which additional detection rules could be introduced in future versions of the toolkit.

7. Windows Registry Forensics

The Windows Registry is a hierarchical database used by the Windows operating system and installed applications to store configuration information. It contains information relating to users, applications, hardware, services and operating system behavior.

From a digital forensics perspective, the Registry can also have valuable evidence about activity that has occurred on a system. Registry artifacts may provide information about previously connected devices, executed applications, recently accessed files and programs configured to execute automatically.

For the 1st version of the Windows Forensics Toolkit, Registry analysis focused specifically on identifying applications configured to execute automatically when a user logs into Windows.

This was achieved by going through the Registry location:
HKCU\Software\Microsoft\Windows\CurrentVersion\Run

7.1 Why the Run Key was Selected

The Windows Run registry key allows applications to be configured to start automatically when a user logs into their account.

Legitimate applications frequently use this functionality. For example, applications such as cloud storage clients, communication software, and other background applications may configure themselves to start automatically.

When the affected user subsequently logs into Windows, the configured executable may be launched automatically.

This shows a form of persistence, where an attacker attempts to make sure that their program continues to execute following a logout, login or system restart.

That is why it is important to note that the presence of a Run-key entry does not automatically show malicious activity. Instead, unusual entries should be treated as evidence requiring further investigation.

7.3 Enumerating Registry Values

Once the Registry key has been opened, the toolkit determines how many values are stored within it:

```
number_of_values = winreg.QueryInfoKey(key)[1]
```

It then iterates through each value using:

```
number_of_values = winreg.QueryInfoKey(key)[1]
```

Each discovered startup entry is converted into a structured Python dictionary:

```
entry = {  
    "name": name,  
    "command": value,  
    "registry_path": f"HKCU\\{RUN_KEY_PATH}",  
    "value_type": value_type  
}
```

The individual entries are then added to a list and returned for further processing.

7.4 Handling Missing Registry Keys

The toolkit also includes basic error handling:

```
except FileNotFoundError:  
    pass
```

This prevents the program from terminating if the specified Run key does not exist on a system.

Instead, the collector simply returns an empty list.

This is important because forensic tools should be capable of handling systems where a particular artifact is unavailable without causing the entire collection process to fail.

7.5 Registry Evidence Output

After collection, the Registry entries are displayed by the main program

During testing, the toolkit successfully identified startup applications configured on the test system.

For example:

```
=====
REGISTRY RUN ENTRIES
=====

Name:      OneDrive
Command:   "C:\Users\favou\AppData\Local\Microsoft\OneDrive\OneDrive.exe" /background
Path:      HKCU\Software\Microsoft\Windows\CurrentVersion\Run

Name:      Opera GX Stable
Command:   "C:\Users\favou\AppData\Local\Programs\Opera GX\opera.exe" --autostart
Path:      HKCU\Software\Microsoft\Windows\CurrentVersion\Run
```

The collected Registry evidence is also exported to output/registry_entries.json.

This allows the original results to be retained in a structured format for subsequent analysis.

7.6 Forensic Relevance

During an investigation, discovering an unfamiliar Run-key entry gives an analyst a starting point for further analysis.

For example, a suspicious Run-key entry together with evidence of unusual PowerShell execution and network activity would provide considerably more investigative context than the Registry entry alone.

7.7 Outcome

The registry collection component successfully shows the toolkits ability to collect forensic evidence from a second Windows data source.

Where the Event Log component focuses on recorded system and authentication activity, the Registry component gives information about system and user configuration that may show persistence mechanisms.

The collected Run-key entries also provide the input for the toolkits second detection mechanisms: Registry persistence detection

8. Testing and Results

Testing was carried out throughout the development of the Windows Forensics Toolkit to show that each component could successfully collect, process and analyze Windows forensic artefacts.

The testing process focused on the two main capabilities implemented in Version 1 of the toolkit: Windows Security Event analysis and Registry persistence analysis.

Controlled activity was generated on the Windows test system, and the toolkit was then executed to show whether the corresponding evidence could be collected and whether the detection rules responded as expected.

8.1 Security Event Collection Test

The first test showed that the toolkit could successfully retrieve selected events from the Windows Security Event Log.

The toolkit was executed on the test system and successfully collected events including successful logons, failed logons, logoffs and other selected security events.

The results were summarized within the terminal using the Event ID categories configured in the program.

```
=====
WINDOWS FORENSICS TOOLKIT
=====

Security events collected: 100

Event Summary
-----
4624 Successful Logons: 92
4625 Failed Logons: 0
4634 Logoffs: 4
4648 Explicit Logons: 4
4688 Process Creation: 0
1102 Security Log Clear: 0

Evidence saved:
output/security_events.json
```

The successful collection of these events show that the toolkit could communicate with the Windows Event Log system and retrieve security information for further analysis.

8.2 Failed Login Detection Test

The brute-force detection functionality was tested by deliberately generating multiple failed authentication attempts on the test Windows system.

Windows recorded these attempts as Security Event ID 4625

The toolkit subsequently collected the events and grouped failed authentication attempts according to their source IP address and target username

During controlled testing, the detection threshold was reduced to 2 failed attempts:

```
def detect_brute_force(events, threshold = 2):
```

This allowed the detection functionality to be tested without repeatedly trying passwords against the Windows account.

When the configured threshold was reached, the toolkit successfully made a security alert:

```
=====
SECURITY ALERTS
=====

[MEDIUM] Possible Brute Force
-----
Source IP:      127.0.0.1
Target User:    -
Failed Attempts: 4
```

This test confirmed that the toolkit could not only collect individual Windows events but also correlate multiple events to identify a potentially suspicious pattern.

During the initial test, the detection didn't generate an alert despite multiple Event ID 4625 records being present.

After investigation I figured out that the test events used the loopback IP address: 127.0.0.1

The original detector excluded this address because loopback traffic would normally not represent a remote brute-force attack.

For controlled testing, the loopback exclusion was temporarily removed, allowing the locally generated events to be analysed.

After making this modification, the detection successfully generated the expected alert.

This troubleshooting process showed the importance of understanding the context of collected forensic evidence rather than relying on the presence of a particular Event ID

8.3 Registry Collection Test

When the toolkit was executed, it enumerated the startup entries configured for the current user.

The test system contained legitimate startup entries including applications such as Microsoft OneDrive and Opera GX.

```
=====
REGISTRY RUN ENTRIES
=====

Name:      OneDrive
Command:   "C:\Users\favou\AppData\Local\Microsoft\OneDrive\OneDrive.exe" /background
Path:      HKCU\Software\Microsoft\Windows\CurrentVersion\Run

Name:      Opera GX Stable
Command:   "C:\Users\favou\AppData\Local\Programs\Opera GX\opera.exe" --autostart
Path:      HKCU\Software\Microsoft\Windows\CurrentVersion\Run

Name:      Steam
Command:   "C:\Program Files (x86)\Steam\steam.exe" -silent
Path:      HKCU\Software\Microsoft\Windows\CurrentVersion\Run

Name:      LGHUB
Command:   "C:\Program Files\LGHUB\system_tray\lghub_system_tray.exe" --minimized
Path:      HKCU\Software\Microsoft\Windows\CurrentVersion\Run
```

This shows that the winreg implementation could successfully access the Registry and convert discovered values into structured records for further analysis.

8.4 Registry Persistence Detection Test

A controlled Registry entry was created to test whether the toolkit could identify potentially suspicious persistence behavior.

This test was selected because PowerShell execution from a Registry Run key can warrant further investigation during a forensic examination.

The toolkit was then executed again.

The Registry detection function examined the command associated with each startup entry and identified powershell.exe as one of the interpreters included within its detection rules.

The test entry therefore generated an alert:

```
=====
REGISTRY ALERTS
=====

[MEDIUM] Suspicious Registry Persistence
-----
Name:      utweb
Command:   "C:\Users\favou\AppData\Roaming\utorrent Web\utweb.exe" /MINI
Path:      HKCU\Software\Microsoft\Windows\CurrentVersion\Run

Reasons:
- Executable launched from suspicious location: \AppData\Roaming\
```

The test entry was removed after testing to restore the Registry to its previous state.

The successful alert demonstrated that the toolkit could differentiate between simply collecting Registry evidence and applying detection logic to identify entries that may require investigation.

8.5 Evidence Export Test

The final test verified that collected evidence and generated alerts could be retained after the toolkit finished executing.

The toolkit successfully generated structured JSON files within the output directory:

```
▼ output
{} failed_logons.json
{} registry_alerts.json
{} registry_entries.json
{} security_alerts.json
{} security_events.json
```

The JSON format allows the results to be reviewed manually while also making them suitable for processing by other scripts or security tools.

This is important because forensic information should not exist only as temporary terminal output. Exporting the results allows the evidence generated during an investigation to be retained for later analysis.

8.6 Overall Results

Testing demonstrated that the Windows Forensics Toolkit can successfully collect and analyze two important sources of Windows forensic information: Security Event Logs and Registry Run keys.

The toolkit was able to transform raw Windows data into structured forensic records, export the resulting evidence, and apply detection logic to identify potentially suspicious authentication and persistence activity.

The controlled tests also highlighted the importance of interpreting security alerts within their wider context. Neither repeated failed logins nor the presence of PowerShell within a startup entry proves that malicious activity has occurred. Instead, the alerts generated by the toolkit are intended to highlight activity that should be investigated further by an analyst.

9. Conclusion

The Windows Forensics Toolkit successfully demonstrated how Python can be used to collect and analyze security-relevant information from a Windows system. Version 1 focused on Windows Security Event Logs and Registry Run keys, allowing the toolkit to collect evidence, convert it into structured data and export the results as JSON.

Detection logic was implemented to identify repeated failed login attempts and potentially suspicious Registry persistence. Controlled testing confirmed that both detection mechanisms worked as intended and could highlight activity requiring further investigation.

Although Version 1 uses relatively simple detection rules, it provides a strong foundation for future development. Version 2 could expand the toolkit with browser history, USB history, Prefetch analysis, Sysmon integration, additional detection rules and forensic timeline generation.

Overall, the project provided practical experience in Windows forensics, Python programming, event-log analysis and threat detection, while showing how endpoint evidence can be transformed into useful information for a SOC analyst or forensic investigator.